

Correspondence Computing Foundations

CoLean / CoPU / C-IR, v0.1

Version: v0.1

Project: CoLean / CoPU / C-IR

Executive Thesis

The core contribution of this project is not a Lean agent. Lean is an important verification environment and benchmark surface, but the deeper proposal is a mathematical and computational substrate for next-generation AI reasoning.

The central claim is:

Reasoning is not primarily dense tensor evaluation or one-shot text generation.
Reasoning is structured correspondence search over composable event spaces.

CoLean is the first verification-oriented prototype of this idea. CoPU and C-IR are the broader architecture and intermediate-representation direction.

The Basic Object

The primitive object is a correspondence:

$$X \leftarrow E \rightarrow Y$$

where:

- X is a source space.
- Y is a target space.
- E is an event space.
- each event carries source, target, weight, and evidence.

In ordinary systems, intermediate structure is often collapsed into a vector, matrix, embedding, or prompt. In correspondence computing, the intermediate event space is preserved.

Composition

Given:

$$\begin{aligned} X &\leftarrow E \rightarrow Y \\ Y &\leftarrow F \rightarrow Z \end{aligned}$$

the composed correspondence is built by a fiber join over matching middle objects:

$$X \leftarrow E \times_Y F \rightarrow Z$$

The computational advantage is that only fiber-compatible events are enumerated. This avoids materializing the full dense triple space:

$$|X| * |Y| * |Z|$$

and instead evaluates:

```
sum_y |E_y| * |F_y|
```

This difference is the source of the acceleration observed in the current prototype.

Push-Reduce

After local composition, many paths may share the same endpoint pair. The push-reduce operation aggregates those paths:

```
many local events -> fewer endpoint-level candidate events
```

This is not merely pruning. It preserves evidence and weight information so that successful and failed reasoning attempts can update the graph.

Failure Is Signal

A normal LLM agent often treats failure as a new prompt message. Correspondence computing treats failure as a structured event:

```
failed candidate  
-> failure class  
-> repair action  
-> updated correspondence weight
```

This matters because reasoning systems should learn from every failed proof, failed retrieval, failed rewrite, or failed mapping.

Relation To Lean

Lean is used as a strict verifier:

```
candidate proof -> Lean kernel / Mathlib -> accepted or rejected
```

The current CoLean prototype demonstrates:

```
LLM free Lean generation: 0/4 accepted  
LLM + CoLean candidate ranking: 4/4 accepted
```

This supports the intended division of labor:

```
LLM: proposal and ranking signal  
CoLean: structured correspondence search  
Lean: correctness oracle
```

Lean is therefore a benchmark and verification case study, not the whole project.

Hardware Direction

The long-term hardware idea is CoPU: a correspondence processing unit.

The relevant primitive operations are:

- sparse event storage
- fiber join

- local compose
- push-reduce
- weight update
- verifier-feedback integration

These are different from conventional dense matrix multiplication. A CoPU-like architecture would specialize in moving and composing structured event fibers, not merely multiplying dense tensors.

C-IR

C-IR is the proposed intermediate representation for correspondence computing. It should represent:

- spaces and typed objects
- event fibers
- source and target maps
- weights and evidence labels
- composition plans
- reduction rules
- verifier feedback
- memory updates

The goal is to make reasoning compilation explicit:

```
problem structure
-> correspondence graph
-> C-IR
-> CoPU/runtime execution
-> verified or evaluated result
```

Current Evidence

The current prototype has demonstrated:

```
structured relation benchmark: 8000x candidate-enumeration reduction
scale sweep: 500x -> 72000x reduction under structured sparsity
local LLM free generation: 0/4 accepted
local LLM + CoLean ranking: 4/4 accepted
KakeyaToy.lean: 1 definition, 7 theorems, sorry = 0
```

These are early mechanism-level results. They are not yet a full SOTA claim. The next milestone is benchmark-grade reproducibility.

Benchmark Path

The project should challenge Lean benchmarks only after the harness is stable. The target metrics are:

- Lean accepts final file
- `sorry = 0`

- top-1 / top-k pass rate
- time to first verified proof
- proof repair iterations
- Mathlib retrieval precision
- dependency graph coverage

Once CoLean reaches competitive benchmark results, the graduation project has a strong complete arc:

```
new mathematical foundation  
-> working prototype  
-> verified Lean case study  
-> benchmark challenge  
-> thesis writeup
```

Summary

The foundational idea is:

```
Do not collapse reasoning into a single vector, matrix, or prompt.  
Preserve correspondences, compose them by fibers, reduce with evidence,  
and update the graph from verifier feedback.
```

CoLean is the first public prototype. The core project is correspondence computing as a mathematical basis for future AI reasoning systems and hardware.